```
!pip -q install pandas numpy scikit-learn matplotlib seaborn joblib kagglehub
```

```
import kagglehub
import os
import glob
import pandas as pd
import numpy as np
path = kagglehub.dataset_download("shivamb/real-or-fake-fake-jobposting-prediction")
print("Dataset downloaded to:", path)
csv_files = glob.glob(os.path.join(path, "**", "*.csv"),recursive=True)
print("Available CSV files:", csv_files)
```

```
Using Colab cache for faster access to the 'real-or-fake-fake-jobposting-prediction' dataset.
Dataset downloaded to: /kaggle/input/real-or-fake-fake-jobposting-prediction
Available CSV files: ['/kaggle/input/real-or-fake-fake-jobposting-prediction/fake_job_postings.csv']
```

```
# Automatically locate the original job-postings CSV
csv_path = next(
    (f for f in csv_files
     if "fake_job_postings" in os.path.basename(f).lower()),
    None)

if csv_path is None:
    raise FileNotFoundError(
        "CSV nahi mili. Kaggle se dataset download karke "
        "Colab mein upload karo.")
df = pd.read_csv(csv_path)
print("Dataset shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())
display(df.head())
```

```
Dataset shape: (17880, 18)

Columns:
['job_id', 'title', 'location', 'department', 'salary_range', 'company_profile', 'description', 'requirements', 'benefits', 'telecommut
```

	job_id	title	location	department	salary_range	company_profile	description	requirements	benefits	te
0	1	Marketing Intern	US, NY, New York	Marketing	NaN	We're Food52, and we've created a groundbreaki...	Food52, a fast-growing, James Beard Award-winn...	Experience with content management systems a m...	NaN	
1	2	Customer Service - Cloud Video Production	NZ, , Auckland	Success	NaN	90 Seconds, the worlds Cloud Video Production ...	Organised - Focused - Vibrant - Awesome!Do you...	What we expect from you:Your key responsibilit...	What you will get from usThrough being part of...	
2	3	Commissioning Machinery Assistant (CMA)	US, IA, Wever	NaN	NaN	Valor Services provides Workforce Solutions th...	Our client, located in Houston, is actively se...	Implement pre-commissioning and commissioning ...	NaN	
3	4	Account Executive - Washington DC	US, DC, Washington	Sales	NaN	Our passion for improving quality of life thro...	THE COMPANY: ESRI – Environmental Systems Rese...	EDUCATION: Bachelor's or Master's in GIS, busi...	Our culture is anything but corporate—we have ...	
4	5	Bill Review Manager	US, FL, Fort Worth	NaN	NaN	SpotSource Solutions LLC is a Global Human Cap...	JOB TITLE: Itemization Review ManagerLOCATION:...	QUALIFICATIONS:RN license in the State of Texa...	Full Benefits Offered	

```
# Remove duplicate rows
df = df.drop_duplicates().copy()

# Check target column
assert "fraudulent" in df.columns, ("Dataset mein fraudulent target column nahi mili.")
# Combine text fields
text_columns = [
    "title",
    "company_profile",
    "description",
    "requirements",
    "benefits"]
for col in text_columns:
    if col not in df.columns:
        df[col] = ""
    df[col] = df[col].fillna("").astype(str)
df["combined_text"] = df[text_columns].agg(" ".join, axis=1)
# Clean target labels
```

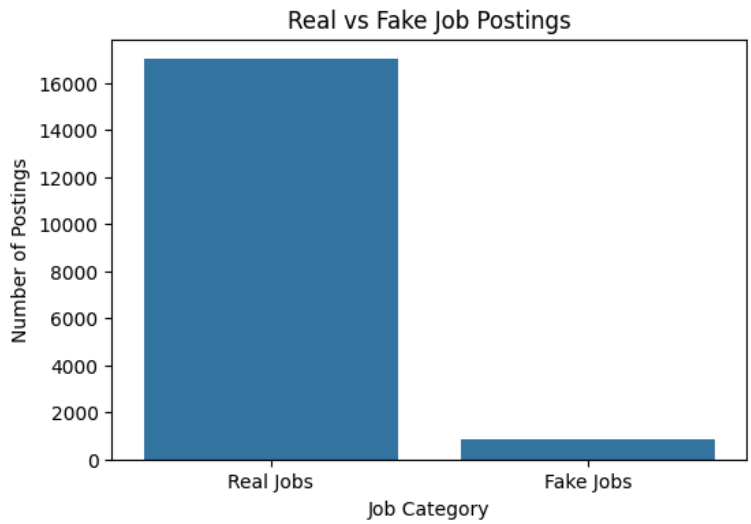
```
df = df[df["fraudulent"].isin([0, 1])].copy()
df["fraudulent"] = df["fraudulent"].astype(int)
# Remove empty job descriptions
df = df[df["combined_text"].str.strip().ne("")].copy()
print("Clean dataset shape:", df.shape)
print("\nTarget distribution:")
print(df["fraudulent"].value_counts())
print("\nFraud percentage:")
print(round(df["fraudulent"].mean() * 100, 2), "%")
```

```
Clean dataset shape: (17880, 19)

Target distribution:
fraudulent
0    17014
1      866
Name: count, dtype: int64

Fraud percentage:
4.84 %
```

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(6, 4))
sns.countplot(
    data=df,
    x="fraudulent")
plt.xticks([0, 1], ["Real Jobs", "Fake Jobs"])
plt.title("Real vs Fake Job Postings")
plt.xlabel("Job Category")
plt.ylabel("Number of Postings")
plt.show()
```




```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

X = df["combined_text"]
y = df["fraudulent"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=1000,
```

```
        class_weight="balanced"
    ),
    "Linear SVM": LinearSVC(
        class_weight="balanced"
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        class_weight="balanced_subsample",
        random_state=42,
        n_jobs=-1
    )
}

results = []
trained_models = {}

for name, classifier in models.items():

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(
            max_features=50000,
            ngram_range=(1, 2),
            stop_words="english",
            sublinear_tf=True
        )),
        ("model", classifier)
    ])

    print(f"\nTraining {name}...")

    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_test)

    if hasattr(pipeline, "predict_proba"):
        scores = pipeline.predict_proba(X_test)[:, 1]
        auc = roc_auc_score(y_test, scores)
    elif hasattr(pipeline, "decision_function"):
        scores = pipeline.decision_function(X_test)
        auc = roc_auc_score(y_test, scores)
    else:
        auc = np.nan

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_test, predictions, zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, predictions, zero_division=0
        ),
        "ROC-AUC": auc
    })

    trained_models[name] = pipeline

    print(classification_report(
        y_test,
        predictions,
        target_names=["Real Job", "Fake Job"],
        zero_division=0
    ))

results_df = pd.DataFrame(results).sort_values(
    by="F1 Score",
    ascending=False
)

display(results_df.round(4))
```

```
Training Logistic Regression...
              precision    recall  f1-score   support

    Real Job       1.00      0.99      0.99      3403
    Fake Job       0.84      0.91      0.87       173

    accuracy                           0.99      3576
   macro avg       0.92      0.95      0.93      3576
weighted avg       0.99      0.99      0.99      3576


Training Linear SVM...
              precision    recall  f1-score   support

    Real Job       0.99      1.00      1.00      3403
    Fake Job       0.97      0.88      0.92       173

    accuracy                           0.99      3576
   macro avg       0.98      0.94      0.96      3576
weighted avg       0.99      0.99      0.99      3576


Training Random Forest...
              precision    recall  f1-score   support

    Real Job       0.98      1.00      0.99      3403
    Fake Job       0.99      0.64      0.77       173

    accuracy                           0.98      3576
   macro avg       0.99      0.82      0.88      3576
weighted avg       0.98      0.98      0.98      3576
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
1	Linear SVM	0.9927	0.9682	0.8786	0.9212	0.9918
0	Logistic Regression	0.9869	0.8351	0.9075	0.8698	0.9900
2	Random Forest	0.9821	0.9910	0.6358	0.7746	0.9924

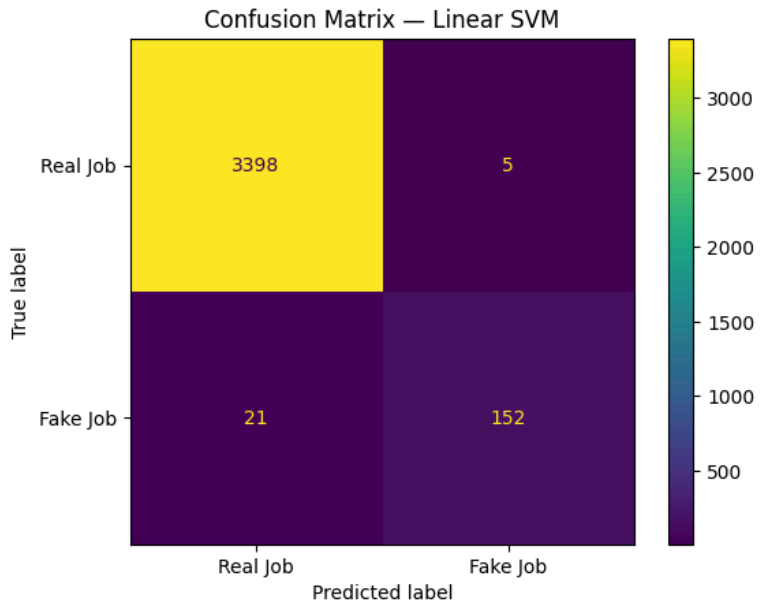
```
best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]

best_predictions = best_model.predict(X_test)

ConfusionMatrixDisplay(
    confusion_matrix=confusion_matrix(
        y_test,
        best_predictions
    ),
    display_labels=["Real Job", "Fake Job"]
).plot()

plt.title(f"Confusion Matrix — {best_model_name}")
plt.show()

print("Best baseline model:", best_model_name)
```




```
Best baseline model: Linear SVM
```

```
job_description = """
We are hiring a Junior Data Analyst.
Candidates should know Python, SQL, Excel and Power BI.
```

```
Applicants should submit their resume through our official
company careers website. No application fee is required.
"""

prediction = best_model.predict([job_description])[0]

print(
    "Prediction:",
    "Potentially Fake / Suspicious Job"
    if prediction == 1
    else "Likely Legitimate Job"
)

print(
    "Note: This is a model prediction, not verification "
    "of the employer or job posting."
)
```

```
Prediction: Likely Legitimate Job
Note: This is a model prediction, not verification of the employer or job posting.
```

```
import joblib
joblib.dump(
    best_model,
    "careershield_ml_model.joblib"
)

results_df.to_csv(
    "careershield_ml_results.csv",
    index=False
)

print("ML model and evaluation results saved.")
```

```
ML model and evaluation results saved.
```

```
import re
import hashlib

def normalize_text(text):
    text = str(text).lower().strip()
    text = re.sub(r"\s+", " ", text)
    return text

df["normalized_text"] = df["combined_text"].apply(normalize_text)

df["text_hash"] = df["normalized_text"].apply(
    lambda x: hashlib.md5(x.encode("utf-8")).hexdigest()
)

print("Total rows:", len(df))
print("Exact duplicate descriptions:", df["text_hash"].duplicated().sum())
print("Unique descriptions:", df["text_hash"].nunique())
```

```
Total rows: 17880
Exact duplicate descriptions: 2008
Unique descriptions: 15872
```

```
df = df.drop_duplicates(
    subset=["text_hash"]
).copy().reset_index(drop=True)

X = df["combined_text"]
y = df["fraudulent"]

print("Rows after duplicate removal:", len(df))
print("\nClass distribution:")
print(y.value_counts())
```

```
Rows after duplicate removal: 15872

Class distribution:
fraudulent
0    15159
1      713
Name: count, dtype: int64
```

```
from sklearn.model_selection import GroupShuffleSplit

# Missing company names ko individual groups dein
company = (
    df["company_profile"]
    .fillna("")
    .astype(str)
    .str.strip()
```

```
    .str.lower()
)

groups = company.copy()

missing = groups.eq("")
groups.loc[missing] = [
    f"unknown_company_{i}"
    for i in df.index[missing]
]

# Keep the split reproducible
splitter = GroupShuffleSplit(
    n_splits=1,
    test_size=0.20,
    random_state=42
)

train_idx, test_idx = next(
    splitter.split(X, y, groups=groups)
)

X_train = X.iloc[train_idx]
X_test = X.iloc[test_idx]
y_train = y.iloc[train_idx]
y_test = y.iloc[test_idx]

train_companies = set(groups.iloc[train_idx])
test_companies = set(groups.iloc[test_idx])

print("Training rows:", len(X_train))
print("Test rows:", len(X_test))
print("Overlapping companies:", len(train_companies & test_companies))
print("Fake-job rate in training:", round(y_train.mean(), 4))
print("Fake-job rate in test:", round(y_test.mean(), 4))

assert len(train_companies & test_companies) == 0
```

```
Training rows: 12861
Test rows: 3011
Overlapping companies: 0
Fake-job rate in training: 0.0376
Fake-job rate in test: 0.0764
```

```
from sklearn.model_selection import StratifiedGroupKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
import pandas as pd
import numpy as np

train_groups = groups.iloc[train_idx]

models = {
    "Logistic Regression": LogisticRegression(
        max_iter=500,
        class_weight="balanced",
        solver="liblinear"
    ),
    "Linear SVM": LinearSVC(
        class_weight="balanced",
        max_iter=1000
    )
}

cv = StratifiedGroupKFold(
    n_splits=3,
    shuffle=True,
    random_state=42
)

results = []

for name, classifier in models.items():
    print("Training:", name, flush=True)

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(
            max_features=15000,
            stop_words="english",
            ngram_range=(1, 1),
            dtype=np.float32
        )),
        ("model", classifier)
    ])
```

```
    scores = cross_validate(
        pipeline,
        X_train,
        y_train,
        groups=train_groups,
        cv=cv,
        scoring=["precision", "recall", "f1", "roc_auc"],
        n_jobs=1
    )

    results.append({
        "Model": name,
        "Precision": scores["test_precision"].mean(),
        "Recall": scores["test_recall"].mean(),
        "F1 Score": scores["test_f1"].mean(),
        "ROC-AUC": scores["test_roc_auc"].mean()
    })

    print("Completed:", name, flush=True)

results_df = pd.DataFrame(results).sort_values(
    "F1 Score", ascending=False
)

display(results_df.round(4))
```

```
Training: Logistic Regression
Completed: Logistic Regression
Training: Linear SVM
Completed: Linear SVM
```

	Model	Precision	Recall	F1 Score	ROC-AUC
1	Linear SVM	0.6529	0.5889	0.6185	0.9224
0	Logistic Regression	0.4248	0.6835	0.5146	0.9282

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)
import pandas as pd

test_results = []

for name, model in trained_models.items():

    predictions = model.predict(X_test)

    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_test)[:, 1]
    else:
        scores = model.decision_function(X_test)

    test_results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, zero_division=0
        ),
        "Recall": recall_score(
            y_test, predictions, zero_division=0
        ),
        "F1 Score": f1_score(
            y_test, predictions, zero_division=0
        ),
        "ROC-AUC": roc_auc_score(y_test, scores)
    })

test_results_df = pd.DataFrame(test_results)

display(test_results_df.round(4))
```

	Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
0	Logistic Regression	0.9904	0.8941	0.9913	0.9402	0.9990
1	Linear SVM	0.9963	0.9740	0.9783	0.9761	0.9998
2	Random Forest	0.9930	1.0000	0.9087	0.9522	0.9996

```
!pip -q install transformers datasets accelerate
```

```
import torch
import numpy as np
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

print("GPU available:", torch.cuda.is_available())

checkpoint = "distilbert/distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(checkpoint)

def prepare_dataset(texts, labels):
    dataset = Dataset.from_dict({
        "text": texts.astype(str).tolist(),
        "labels": labels.astype(int).tolist()
    })

    def tokenize(batch):
        return tokenizer(
            batch["text"],
            truncation=True,
            padding="max_length",
            max_length=128
        )

    dataset = dataset.map(tokenize, batched=True)
    dataset = dataset.remove_columns(["text"])
    dataset.set_format("torch")
    return dataset

train_ds = prepare_dataset(X_train, y_train)
test_ds = prepare_dataset(X_test, y_test)

print("Training samples:", len(train_ds))
print("Test samples:", len(test_ds))
```

```
GPU available: False
```

```
Map: 100%  12861/12861 [00:43<00:00, 297.48 examples/s]
```

```
Map: 100%  3011/3011 [00:07<00:00, 394.52 examples/s]
```

```
Training samples: 12861
Test samples: 3011
```